Cybersecurity & DevSecOps · Abia Cohort 3.0 · 2026

WEEK 9 · DAY 4

Git Security — Secrets Detection and Environment Variables

Student Name: _____

WHY HARDCODED SECRETS ARE DANGEROUS

The Real-World Risk

- The most common cause of cloud breaches is hardcoded credentials accidentally committed to Git
- GitHub automatically scans public repositories for AWS keys, Stripe keys, and other secrets
- Once a key is committed — even if deleted in the next commit — it lives in Git history forever
- Attackers run automated bots that scan GitHub 24/7 looking for newly committed secrets
- A leaked AWS key can generate cloud bills of $50,000 or more in hours before you notice
- In 2022, Samsung, Twitch, and Toyota all suffered breaches caused by exposed secrets in code

Everything you will do today is about prevention — finding secrets before they reach Git, and writing code that never stores credentials in the first place.

LAB PART A — FIND HARDCODED SECRETS

STEP 1	Install the Tools

BASH

```
# Install TruffleHog — automated secrets detection
pip3 install truffleHog --break-system-packages

# Verify installation
truffleHog --version

# Also install git-secrets for pre-commit scanning
sudo apt install git-secrets -y 2>/dev/null || \
 git clone https://github.com/awslabs/git-secrets.git \
 ~/tools/git-secrets
```

STEP 2	Create a Test Repository With Hidden Secrets

You are going to create a repository that simulates what a developer accidentally commits — real-looking but fake credentials in config files:

BASH — STEP 1: Initialise the repository

```
# Create the lab directory and initialise Git
mkdir -p ~/labs/week9/git_security/payliteng_app
cd ~/labs/week9/git_security/payliteng_app
```



```
git init
git config user.email 'student@techrise.ng'
git config user.name 'TechRise Student'
```

BASH — STEP 2: Create config.py with secrets

```
# Create a Python config file with hardcoded secrets
cat > config.py << 'EOF'
# PayliteNG application configuration
# Author: Dev Team
# Last updated: January 2025

DATABASE_HOST = 'prod-db.payliteng.internal'
DATABASE_PORT = 3306
DATABASE_USER = 'paylite_root'
DATABASE_PASS = 'PayliteDB@2024!'
DATABASE_NAME = 'paylite_production'

# Payment gateway credentials
FLUTTERWAVE_SECRET = 'FLWSECK-a1b2c3d4e5f6g7h8-X'
PAYSTACK_SECRET_KEY = 'sk_live_9f8e7d6c5b4a3b2c1'

# AWS credentials for file storage
AWS_ACCESS_KEY_ID = 'AKIAIOSFODNN7EXAMPLE'
AWS_SECRET_ACCESS_KEY = 'wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY'
AWS_BUCKET_NAME = 'payliteng-customer-backups'

# Application settings
DEBUG = True
SECRET_KEY = 'django-insecure-abc123xyz789'
ALLOWED_HOSTS = ['*']
EOF
```

BASH — STEP 3: Create database.php with credentials

```
# Create a PHP database connection file with hardcoded credentials
cat > database.php << 'EOF'
<?php
// PayliteNG database connection
// WARNING: Do not share this file

$host = 'localhost';
$user = 'root';
$password = 'paylite2024';
$database = 'paylite_prod';
```

```
$conn = mysqli_connect($host, $user, $password, $database);

if (!$conn) {
 die('Connection failed: ' . mysqli_connect_error());
}
?>
EOF
```

BASH — STEP 4: Commit to Git

```
# Commit both files to Git — simulating a developer mistake
git add .
git commit -m 'Initial commit — PayliteNG application config'

# Check what is now in Git history
git log --oneline
```

✓ Files committed to Git

```
a1b2c3d Initial commit — PayliteNG application config

← Files are now in Git history — even if deleted later they remain here
```


STEP 3	Scan With TruffleHog

Now scan the repository for secrets:

BASH — TRUFFLEHOG SCAN

```
# Method 1 — Scan the Git repository with TruffleHog
cd ~/labs/week9/git_security
truffleHog file:///home/kali/labs/week9/git_security/payliteng_app \
 --json 2>/dev/null | python3 -m json.tool
```

✓ TruffleHog detected secrets

```
{
  "reason": "High Entropy String",
  "path": "config.py",
  "stringsFound": [
    "AKIAIOSFODNN7EXAMPLE",
    "FLWSECK-a1b2c3d4e5f6...",
    "sk_live_9f8e7d6c5b4a..."
  ]
}

← TruffleHog found the secrets automatically
```

STEP 4	Manual Scan With grep

grep is the simplest secrets scanner — every developer should know this command before pushing any code:

BASH — GREP SCAN

```
cd ~/labs/week9/git_security/payliteng_app

# Scan all Python and PHP files for common secret patterns
grep -rn --include='*.py' --include='*.php' --include='*.js' \
 --include='*.env' --include='*.config' \
 -E 'password|passwd|secret|api_key|access_key|token|credential|private_key'
\ . | grep -v '^#'
```

✓ grep found all hardcoded secrets

```
config.py:7:DATABASE_PASS = 'PayliteDB@2024!'
config.py:11:FLUTTERWAVE_SECRET = 'FLWSECK-a1b2c3d4e5f6g7h8-X'
config.py:12:PAYSTACK_SECRET_KEY = 'sk_live_9f8e7d6c5b4a3b2c1'
config.py:15:AWS_ACCESS_KEY_ID = 'AKIAIOSFODNN7EXAMPLE'
config.py:16:AWS_SECRET_ACCESS_KEY = 'wJalrXUtn...'
database.php:7:$password = 'paylite2024';

← 6 secrets found across 2 files — file and line number shown
```

BASH — SCAN GIT HISTORY

```
# Also search Git HISTORY — finds secrets in old commits
git log -p | grep -E 'password|secret|key|token' | head -20
```

🔍 git log -p shows every change in every commit. If a secret was in a commit three months ago and 'deleted' in the next commit — it is still in the history. Attackers know this.

STEP 5 Record What You Found

BASH — DOCUMENT FINDINGS

```
# Save your findings
cat > ~/labs/week9/git_security/secrets_found.txt << 'EOF'
```

```
=== SECRETS SCAN RESULTS ===
Repository: payliteng_app
Analyst: [YOUR NAME]
Date: [TODAY]
Tool: TruffleHog + grep

SECRETS FOUND:
1. DATABASE_PASS — config.py line 7
   Risk: Database compromised if leaked

2. FLUTTERWAVE_SECRET — config.py line 11
   Risk: Payment transactions can be made in PayliteNG name

3. PAYSTACK_SECRET_KEY — config.py line 12
   Risk: Payments charged to PayliteNG account

4. AWS_ACCESS_KEY_ID — config.py line 15
   Risk: Full AWS account access — potential $50,000+ bills

5. AWS_SECRET_ACCESS_KEY — config.py line 16
   Risk: Same as above — both keys needed for AWS access

6. $password — database.php line 7
   Risk: Database accessible to anyone with this password

RECOMMENDATION: Rotate ALL credentials immediately.
  Remove from Git history with git-filter-repo.
  Switch to environment variables (see Part B).
EOF

cat ~/labs/week9/git_security/secrets_found.txt
```

✓ Part A Complete — Secrets Found
☐ TruffleHog installed and version confirmed
☐ Test repository created and both files committed
☐ TruffleHog scan ran — secrets detected in output
☐ grep scan ran — 6 secrets found with file and line numbers
☐ Git history scan ran — secrets visible in old commits
☐ Findings documented in secrets_found.txt

LAB PART B — THE FIX: ENVIRONMENT VARIABLES

Now that you have found the secrets — fix them. The correct approach is to remove all credentials from code and replace them with environment variables. This way the code is safe to commit but the credentials stay on the server only.

STEP 1	Understand What an Environment Variable Is

Environment Variables Explained

- An environment variable is a named value stored in the operating system — not in the code
- Set once on the server, accessible by any program running on that server
- Never committed to Git — stays on the machine only
- If the code is stolen — no credentials are inside it
- In Python: os.environ.get('VARIABLE_NAME') reads the value
- In PHP: getenv('VARIABLE_NAME') or $_ENV['VARIABLE_NAME']

STEP 2 Rewrite config.py Using Environment Variables

BASH — REWRITE config.py

```
cat > ~/labs/week9/git_security/payliteng_app/config_secure.py << 'EOF'
import os

# PayliteNG configuration — SECURE VERSION
# All credentials loaded from environment variables
# This file is SAFE to commit to Git

DATABASE_HOST = os.environ.get('DB_HOST', 'localhost')
DATABASE_PORT = int(os.environ.get('DB_PORT', '3306'))
DATABASE_USER = os.environ.get('DB_USER')
DATABASE_PASS = os.environ.get('DB_PASS') # no default — must be set
DATABASE_NAME = os.environ.get('DB_NAME')

# Payment gateway
FLUTTERWAVE_SECRET = os.environ.get('FLUTTERWAVE_SECRET')
PAYSTACK_SECRET_KEY = os.environ.get('PAYSTACK_SECRET_KEY')

# AWS credentials
AWS_ACCESS_KEY_ID = os.environ.get('AWS_ACCESS_KEY_ID')
AWS_SECRET_ACCESS_KEY = os.environ.get('AWS_SECRET_ACCESS_KEY')
AWS_BUCKET_NAME = os.environ.get('AWS_BUCKET_NAME')

# Application settings
DEBUG = os.environ.get('DEBUG', 'False') == 'True'
SECRET_KEY = os.environ.get('DJANGO_SECRET_KEY')
ALLOWED_HOSTS = os.environ.get('ALLOWED_HOSTS', 'localhost').split(',')
EOF

echo 'Secure config file created'
cat ~/labs/week9/git_security/payliteng_app/config_secure.py
```

 Notice os.environ.get('DB_PASS') has NO default value. If the environment variable is not set, Python returns None and the application will crash — this is intentional. It forces developers to set credentials before deploying.

STEP 3 Create the .env File — Local Credentials Store

The .env file stores credentials on the local machine. It must NEVER be committed to Git:

BASH — CREATE .env FILE

```
# Create the .env file with the actual credentials
cat > ~/labs/week9/git_security/payliteng_app/.env << 'EOF'
# PayliteNG environment variables
# DO NOT COMMIT THIS FILE TO GIT

DB_HOST=prod-db.payliteng.internal
DB_PORT=3306
DB_USER=paylite_root
DB_PASS=PayliteDB@2024!
DB_NAME=paylite_production

FLUTTERWAVE_SECRET=FLWSECK-a1b2c3d4e5f6g7h8-X
PAYSTACK_SECRET_KEY=sk_live_9f8e7d6c5b4a3b2c1

AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
AWS_BUCKET_NAME=payliteng-customer-backups

DEBUG=False
DJANGO_SECRET_KEY=new-secure-random-key-here
ALLOWED_HOSTS=payliteng.com,www.payliteng.com
EOF
```



```
echo '.env file created'
```

STEP 4	Add .env to .gitignore — CRITICAL

⚠ Never Commit .env to Git

Without a .gitignore entry, git add . will include the .env file in the next commit — defeating the entire purpose. This step is NOT optional.

BASH — CREATE .gitignore

```
# Create .gitignore to exclude sensitive files
cat > ~/labs/week9/git_security/payliteng_app/.gitignore << 'EOF'
# Environment and credentials — NEVER commit these
.env
*.env
.env.local
.env.production

# Old insecure config — replace with config_secure.py
config.py
database.php

# Python
__pycache__/
*.pyc
*.pyo

# Common IDE and OS files
```

```
.DS_Store
*.swp
*.swo
EOF

cat .gitignore
```

STEP 5 Verify the Fix Works

BASH — TEST ENVIRONMENT VARIABLES

```
cd ~/labs/week9/git_security/payliteng_app

# Load the .env variables into the current shell
export $(cat .env | grep -v '#' | xargs)

# Test that Python can read them
python3 -c "
import os
print('DB_HOST:', os.environ.get('DB_HOST'))
print('DB_USER:', os.environ.get('DB_USER'))
print('DB_PASS:', os.environ.get('DB_PASS'))
print('AWS_KEY:', os.environ.get('AWS_ACCESS_KEY_ID'))
"
```

✓ Environment variables working

```
DB_HOST: prod-db.payliteng.internal
DB_USER: paylite_root
DB_PASS: PayliteDB@2024!
AWS_KEY: AKIAIOSFODNN7EXAMPLE

← Python reads credentials from environment — NOT from code
← config_secure.py can now be safely committed to Git
```

STEP 6 Commit the Secure Version

BASH — COMMIT THE FIX

```
cd ~/labs/week9/git_security/payliteng_app

# Add only the safe files
git add config_secure.py .gitignore

# Verify .env is NOT staged
git status

# Commit
git commit -m 'Fix: replace hardcoded secrets with environment variables'

# Confirm .env is ignored
git ls-files .env
# Should return nothing — .env is not tracked
```

✓ Only safe files committed

```
On branch master
Changes to be committed:
  new file: config_secure.py
  new file: .gitignore

Untracked files (not staged for add):
   .env ← .env is here — not in 'Changes to be committed'

← .env is untracked and will not be committed
```

STEP 7	IMPORTANT — Remove the Old Secrets From Git History

⚠ Deleting config.py is NOT enough

config.py with all its credentials is still in the very first commit. Anyone can run git log -p and read every credential. You must rewrite Git history to remove it.

BASH — REMOVE SECRETS FROM GIT HISTORY

```
# Install git-filter-repo (the correct tool for rewriting history)
pip3 install git-filter-repo --break-system-packages

# Remove config.py from ALL Git history
cd ~/labs/week9/git_security/payliteng_app
git filter-repo --path config.py --invert-paths --force

# Remove database.php from ALL Git history
git filter-repo --path database.php --invert-paths --force

# Verify the files are gone from history
git log -p | grep -E 'DATABASE_PASS|FLUTTERWAVE|AWS_ACCESS' | wc -l
# Should return 0
```

✓ History cleaned

```
Parsed 2 commits
New history written in 0.01 seconds
0

← config.py removed from all commits
← Secrets no longer visible anywhere in Git history
```

After cleaning Git history — rotate ALL credentials immediately:

- Change the database password — even though the file is gone, anyone who already cloned the repo has the old password
- Regenerate Flutterwave and Paystack keys in their dashboards

- Delete and recreate the AWS access keys in the IAM console
- Generate a new Django SECRET_KEY

✓ Part B Complete — Secrets Fixed
☐ config_secure.py written using os.environ.get() for all credentials
☐ .env file created with actual credential values
☐ .gitignore created — .env and config.py listed
☐ Environment variables tested — Python prints correct values
☐ git status confirms .env is NOT staged
☐ Safe files committed: config_secure.py + .gitignore
☐ git-filter-repo removed config.py and database.php from all history
☐ git log -p returns 0 matches for secrets
☐ Understanding: would rotate all credentials in a real scenario

QUICK REFERENCE — BEFORE AND AFTER



	✗ WRONG — Hardcoded Secret	✓ CORRECT — Environment Variable
Python	`DATABASE_PASS = 'secret123'`	`DATABASE_PASS = os.environ.get('DB_PASS')`
PHP	`$password = 'secret123';`	`$password = getenv('DB_PASS');`
Node.js	`const pass = 'secret123'`	`const pass = process.env.DB_PASS`
Git risk	`Credential in every clone`	`No credential in the code`
If repo leaked	`Database immediately at risk`	`Attacker gets no credentials`

FINAL CHECKLIST — DAY 4 COMPLETE



✓ All items complete — Day 4 Git Security Lab finished
☐ TruffleHog installed and working
☐ Practice repository created with two files containing secrets
☐ TruffleHog scan detected secrets automatically

☐ grep scan found all 6 secrets with file and line numbers
☐ Git history scan confirmed secrets persist in old commits
☐ config_secure.py written — all credentials use os.environ.get()
☐ .env file created locally — contains actual credentials
☐ .gitignore excludes .env from all future commits
☐ Python confirmed reading credentials from environment
☐ git status confirmed .env NOT staged for commit
☐ git-filter-repo removed secrets from all Git history
☐ Can explain why deleting a file does not remove it from Git history
☐ Can explain why rotating credentials is necessary after a leak

Environment variables cost nothing to implement and prevent some of the most common breaches in the world.